Group Project: Sorting

This project will utilize the following sorting algorithms:

- Selection Sort, Insertion Sort, Shell Sort, Bubble Sort, Heap Sort, Merge Sort, Quick Sort, Radix Sort, and Counting Sort.
- Advanced exploration: Binary Insertion Sort, Shaker Sort, Flash Sort.

Details about the requirements can be found in the information below.

1. Programming

1.1 Algorithms

You are tasked with implementing all the above sorting algorithms in C/C++ for **ascending order**.

1.2 Experiments

For this project, the following scenario outlines the necessary steps for conducting the experiments:

```
for each Data Order S1:
  for each Data Size S2:
    - Create an original array A1 with Data Order S1 and Data Size S2
    for each Sorting Algorithm S3:
      - Make a duplicate array A2 of the original array A1
      - Sort array A2 using the Sorting Algorithm S3, while:
        + Measuring the running time (in millisecs), and
        + Counting the number of comparisons in the algorithm
      - Take note of S1, S2, S3, running time and number of comparisons
    end for
  end for
end for
```

Data Order: Evaluate the sorting algorithms on various data arrangements including **sorted** (in ascending order), **nearly sorted**, **reverse sorted**, and **randomized** data.
Refer to `DataGenerator.cpp` for more information.
Data Size: Evaluate the algorithms on datasets of varying sizes: 10,000, 30,000, 50,000, 100,000, 300,000, and 500,000 elements.

1.3 Output specifications

Your source code must be compiled into a single executable supporting algorithm and comparison modes. Use this command to compile:

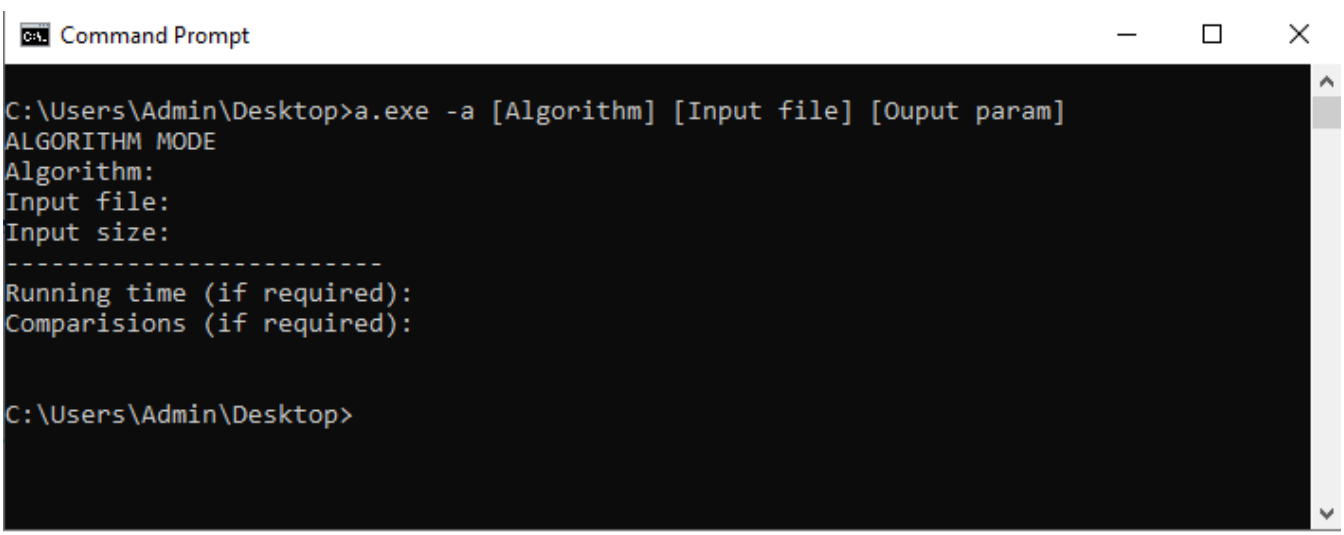
```
> g++ *.cpp -std=c++14 -o groupid.[exe]
```

Algorithm mode: This mode allows you to execute a specific sorting algorithm on input data. This data can be either user-provided or automatically generated. The mode then outputs the measured execution time and/or the number of comparisons performed by the algorithm.
The prototype of this mode is:

```
> groupid.[exe] -mode Algorithm Input_file/Input_size Input_order Output_parameter

OPTIONS:
  -mode             -a algorithm mode, -c comparison mode
  Algorithm         selection-sort, insertion-sort,...
  Input_file        path to user-provided data, with format:
                      1st line: integer n (number of elements)
                      2nd line: n integers, separated by a single space
  Input_size        generated data with specified size, Integer (≤ 1,000,000)
  Input_order       generated data with order: -sorted, -nsorted, -rev, -rand
  Output_parameter  -time or -comp or -both
```

- *Command 1*: Run a sorting algorithm on user-provided data

```
# Prototype:
> groupid.[exe] -a Algorithm Input_file Output_parameter
# For example:
> 01.[exe] -a radix-sort input.txt -both
```



- *Command 2*: Run on the data generated automatically with specified size and order

```
# Prototype:
> groupid.[exe] -a Algorithm Input_size Input_order Output_parameter
# For example:
> 01.[exe] -a selection-sort 50 -rand -time
```

```
C:\Users\Admin\Desktop>a.exe -a [Algorithm] [Input size] [Input order] [Ouput param]
ALGORITHM MODE
Algorithm:
Input size:
Input order:
-------------------------
Running time (if required):
Comparisions (if required):


C:\Users\Admin\Desktop>
```

- *Command 3*: Run a sorting algorithm on ALL data arrangements of a specified size

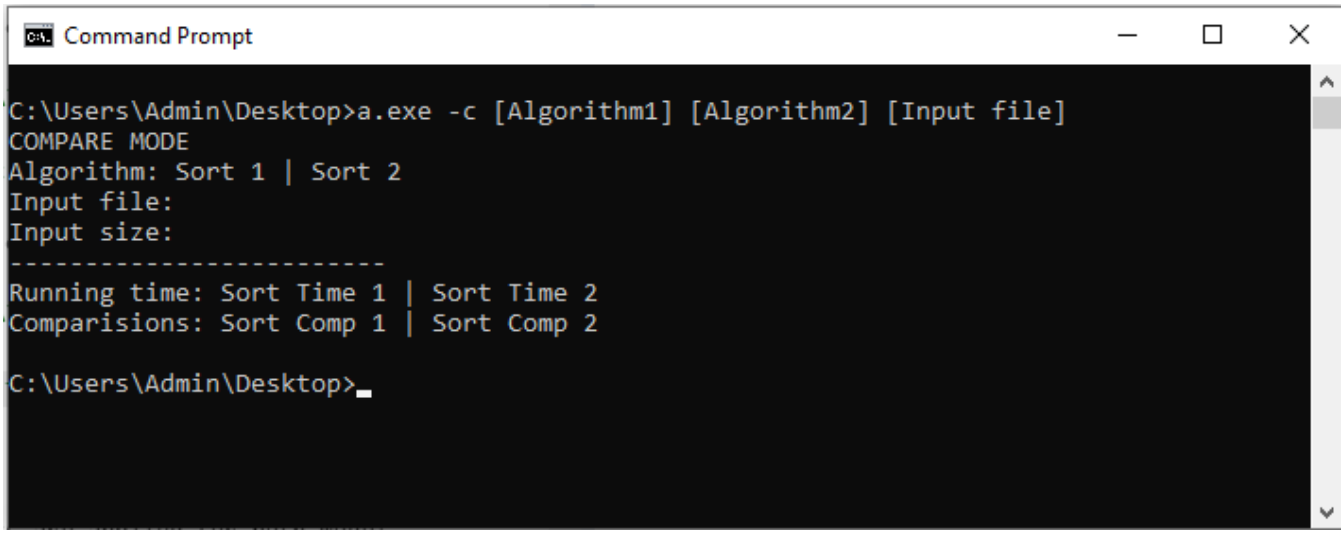
```
# Prototype:
> groupid.[exe] -a Algorithm Input_size Output_parameter
# For example:
> 01.[exe] -a quick-sort 70000 -comp
```

```
C:\Users\Admin\Desktop>a.exe -a [Algorithm] [Input size] [Output param]
ALGORITHM MODE
Algorithm:
Input size:

Input order: Randomize
-------------------------
Running time (if required):
Comparisions (if required):

Input order: Nearly Sorted
-------------------------
Running time (if required):
Comparisions (if required):

Input order: Sorted
-------------------------
Running time (if required):
Comparisions (if required):

Input order: Reversed
-------------------------
Running time (if required):
Comparisions (if required):


C:\Users\Admin\Desktop>
```

Comparison mode: This mode allows you to compare two sorting algorithms on input data. This data can be either user-provided or automatically generated. The mode then outputs the measured execution times and/or the number of comparisons performed by each algorithm

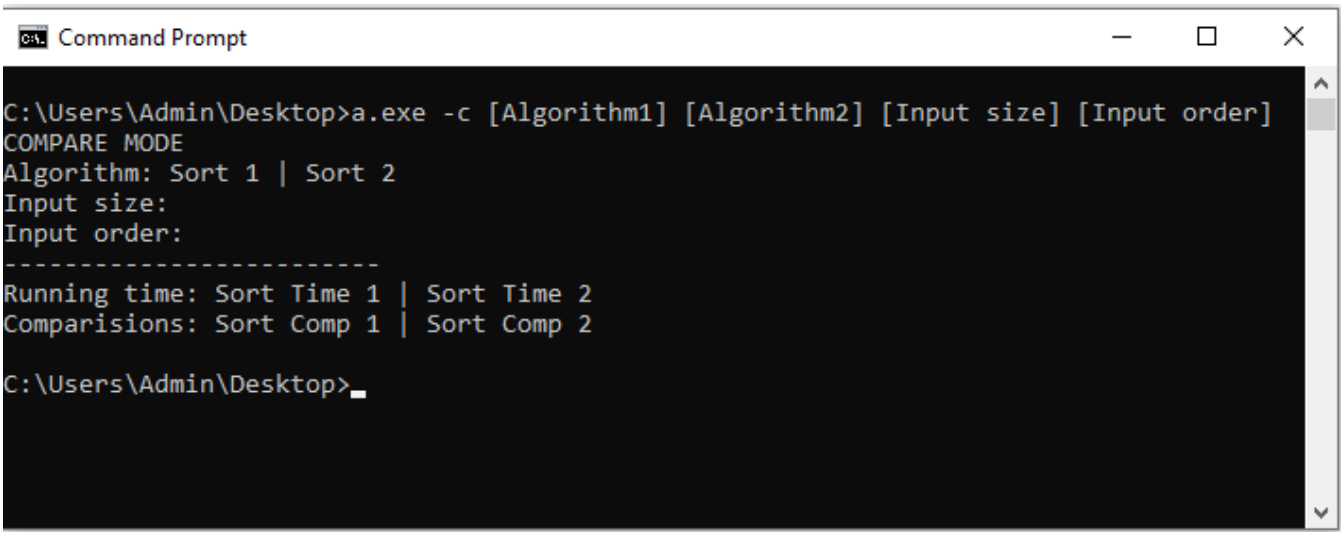
- *Command 4*: Run two sorting algorithms on user-provided data

```
# Prototype:
> groupid.[exe] -c Algorithm_1 Algorithm_2 Input_file
# For example:
> 01.[exe] -c heap-sort merge-sort input.txt
```



- *Command 5*: Run two sorting algorithms on the data generated automatically with a specified size and order

```
# Prototype:
> groupid.[exe] -c Algorithm_1 Algorithm_2 Input_size Input_order
# For example:
> 01.[exe] -c quick-sort merge-sort 100000 -nsorted
```



Writing files: In addition to the console output described above, you are required to write down the corresponding input(s) or output(s).

- For *Command 1* and *Command 2*: Write down the sorted array to the "output.txt" file.
- For *Command 2* and *Command 5*: Write down the generated input to the "input.txt" file.

- For *Command 3*: Write down all four generated input: "input_1.txt" as random order data, "input_2.txt" as nearly sorted data, input_3.txt" as sorted data, "input_4.txt" as reversed data.

2. Report

Structure your report file with the following sections:

2.1 Information page

You must include your group's student information and a table showing each individual's contribution.

Student Name	ID	Primary Responsibilities	Contribution %
[Name 1]	[ID 1]	Implementation (all sorts) + Experimental (all data) + command line arguments (all 5)	60%
[Name 2]	[ID 2]	Report (full)	31%
[Name 3]	[ID 3]	Report (information page)	5%
[Name 4]	[ID 4]		0%
Total			**97%**

2.2 Contents page

2.3 Algorithm presentation

This section delves into the implemented sorting algorithms. Here, you'll provide a comprehensive breakdown for each algorithm, including:

- Core Concepts: Briefly discuss the core ideas behind each algorithm.
- Step-by-step Explanations: Provide a clear and step-by-step explanation for each algorithm, including examples for more details.
- Complexity Analysis: Analyze the time complexity of each algorithm. If applicable, also discuss the space complexity.
- Variants and Optimizations: If any of the algorithms have known variants or optimizations, discuss them here.

2.4 Experimental results and comments

- You are required to organize the experimental results into FOUR tables, each representing one **Data order**. In each table, present the resulting statistics (i.e., running times or numbers of comparisons) of all sorting algorithms following a specific data arrangement. The table template is shown below:

Data order: ...						
Data size	10,000		50,000		...	
Resulting statics	Running time	Comparision	Running time	Comparision	Running time	Comparision
Sorting algorithm 1						
Sorting algorithm 2						
...						

- You are also required to make visualization by graphs:

- There will be four LINE GRAPHs, each of which corresponds to a table of running times. In every graph, the horizontal axis is for `Data Size` and the vertical axis is for running time, as shown in Figure 1.

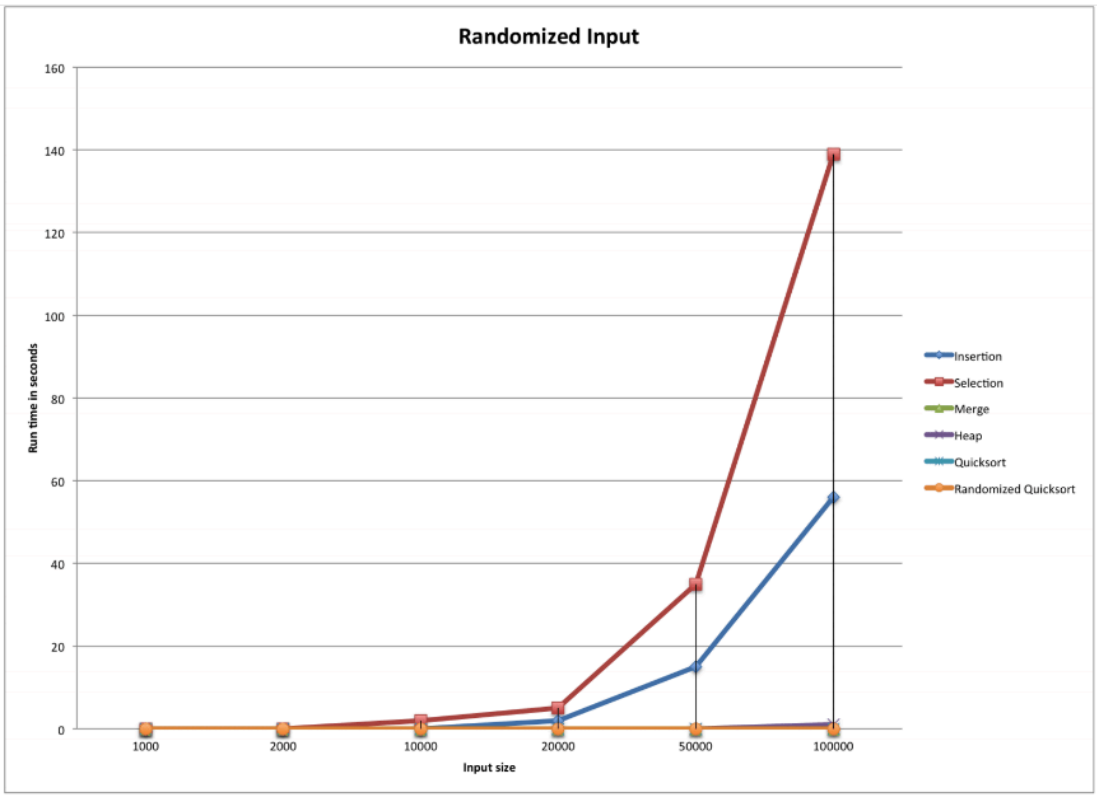


Figure 1: An example of a line graph for visualizing the algorithms' running times on randomized input data.

- There will be four BAR CHARTS, each of which corresponds to a table of numbers of comparisons. In every graph, the horizontal axis is for `Data Size`, and the vertical axis is for the number of comparisons, as shown in Figure 2.

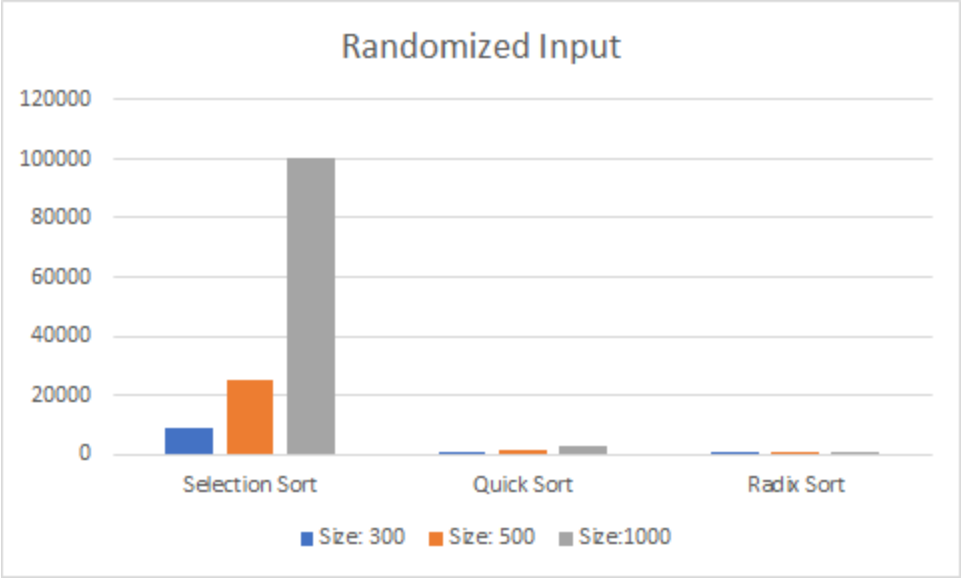


Figure 2: An example of a bar chart for visualizing the algorithms' number of comparisons on randomized input data.

- Make comments based on your own observations on each graph (e.g., the fastest/slowest or the most/least comparisons algorithm(s) in each case, time or comparisons acceleration of algorithms, etc.). Explain your comments.
- Make an overall comment on algorithms on all `Data Order` and all `Data Size` (the fastest/slowest algorithms overall, grouping the stable/unstable algorithms, etc.)

2.5 Project organization and Programming notes

A brief explanation of how you organized your source codes, and notes of any special libraries[2,3]/data structures[1,2] used.

2.6 References

1. **Author Name**, "Book Title", Reference Chapter, Publisher, Year of Publication.
2. **Author Name**, "Article Title", Conference/Journal Name, Year of Publication.
3. **Author Name**, "Article/Webpage Title", URL, Accessed Date.
4. **Author Name**, "Slide/Lecture Note Title" [PowerPoint Slides/Lecture Notes], URL.

2.7 Acknowledgements :

Support was provided by Mr./Ms./[Relationship] ABCD in developing the code for the XYZ function, among others.

In addition, AI tools, such as ChatGPT/Gemini/Deepseek, assisted with drafting Section I of the report and implementing the XYZ function.

3. Submission

This is a 4-person group project. Formation of groups with fewer than 4 members requires permission from the instructors. Individual assignments will not be accepted.

- Create the folder to include the following materials:
  - *source* folder: Contains the project's source code files, only files of extensions **.cpp** and **.h** are required.
  - *report.pdf*: Project report. Only files with the .pdf extension are allowed.
  - *README.txt*: Contains notes on the libraries used. DO NOT include libraries that are not part of the GNU standard.
- Compress the above folder into a file of extension **zip** and name it following your Group ID (e.g.01.zip).
- Only one member representing the group submits the assignment.

Submissions that violate any regulations will get a score of ZERO.

Plagiarism and Cheating will result in a score of ZERO for the entire course and will be subject to appropriate referral to the Management Board for further action.

--- THE END ---